

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования
"Омский государственный технический университет"

Кафедра «Автоматизированные системы обработки информации и управления»

Отчет

По лабораторным работам 6 - 10

дисциплина «Объектно-ориентированное программирование»

Выполнил(а): Шмидт А.В. гр. ИВТ - 244

Проверил: доцент, к.т.н Никонов В.А.

Омск 2026

Лабораторная работа №6

Задание:

- 1) Написать интерфейс IConsole содержащий:
 1. Тип процессора (Сделать Enum);
 2. Название производителя (Сделать Enum);
 3. Тип носителей для консоли (CD, DVD, Картриджи и т.д.) (Сделать Enum);
 4. Модель (зависит от производителя);
 5. Объём жесткого диска (int);
 6. Установленные игры (List<string>);
 7. Аккаунты системы (List <string>).
- 2) Написать интерфейс IHack содержащий (продумать реализацию и тип возвращаемого значения):
 1. Метод TrySoftwareHack();
 2. Метод TryHardwareHack().
- 3) Написать класс Console:
 1. Реализующий интерфейсы IConsole и IHack;
 2. Конструкторы по умолчанию и с параметрами;
 3. Статический метод Generate возвращающий экземпляр класса Console со случайным набором параметров;
 4. Статический метод Generate100 возвращающий 100 экземпляров класса Console со случайным набором параметров.
- 4) Создать коллекцию List<Console>
 1. Поместить в нее результат работы метода Generate100;
 2. Отфильтровать по Типу процессора;
 3. Отфильтровать по Типу процессора и названию производителя;
 4. Отфильтровать по коллекции пользователей и объёму оперативной памяти.

- 5) Создать коллекцию List<Console>
 1. Поместить в нее результат работы метода Generate100;
 2. Отсортировать по Типу процессора;
 3. Отсортировать по Типу процессора и названию производителя.
- 6) Создать коллекцию List<Console >
 1. Поместить в нее результат работы метода Generate100;
 2. Написать SELECT который возвращает коллекцию, содержащую следующие поля;
 3. Тактовая частота процессов (int);
 4. Объём жесткого диска (int);
 5. Аккаунты системы (List <string>).
- 7) Написать интерфейс IManufacturer содержащий данные о производителях:
 1. Название;
 2. Страна (Сделать Enum);
 3. Количество сотрудников;
 4. Написать класс Manufacturer.
- 8) Реализующий интерфейс IManufacturer.
 1. Конструкторы по умолчанию и с параметрами;
 2. Создать и заполнить данными коллекцию List<Manufacturer>;
 3. Показать, как работает внутреннее соединение.
- 9) Написать расширяющий метод для типа string, удаляющий латинские буквы.
- 10) Переписать задания 2-5 используя расширяющие методы LINQ.

Код:

```
IConsole.cs

public interface IConsole
{
    ProcessorType Processor { set; get; }
}
```

```

int CpuClockMHz { get; set; }
ManufacturerName ManufacturerName { set; get; }
MediaType Media { set; get; }
string Model { set; get; }
int RAMSizeGb { get; set; }
int HardDriveSizeGb { set; get; }
IReadOnlyList<string> InstalledGames { get; }
IReadOnlyList<string> Accounts { get; }
}

```

IHack.cs

```

public interface IHack
{
    bool TrySoftwareHack();
    bool TryHardwareHack();
}

```

IManufacturer.cs

```

public interface IManufacturer
{
    string Name { get; set; }
    Country Country { get; set; }
    int EmployeeCount { get; set; }
}

```

Console.cs

```

public class GameConsole : IConsole, IHack, ICloneable
{
    private EventHandler<string>? _userAdded;
    private EventHandler? _driveConnected;
    private EventHandler<string>? _gameInstalled;
    private EventHandler? _gamepadReplaced;

    public event EventHandler<string> UserAdded
    {
        add { _userAdded += value; }
        remove { _userAdded -= value; }
    }

    public event EventHandler DriveConnected
    {
        add { _driveConnected += value; }
        remove { _driveConnected -= value; }
    }

    public event EventHandler<string> GameInstalled
    {

```

```

    add { _gameInstalled += value; }
    remove { _gameInstalled -= value; }
}

public event EventHandler GamepadReplaced
{
    add { _gamepadReplaced += value; }
    remove { _gamepadReplaced -= value; }
}

public ProcessorType Processor { get; set; }
public int CpuClockMHz { get; set; }
public ManufacturerName ManufacturerName { get; set; }
public MediaType Media { get; set; }
public string Model { get; set; }
public int RAMSizeGb { get; set; }
public int HardDriveSizeGb { get; set; }

private readonly List<string> _installedGames = [];
public IReadOnlyList<string> InstalledGames => _installedGames;

private readonly List<string> _accounts = [];
public IReadOnlyList<string> Accounts => _accounts;

public GameConsole()
{
    Processor = ProcessorType.X64;
    CpuClockMHz = 1800;
    ManufacturerName = ManufacturerName.Microsoft;
    Media = MediaType.Flash;
    Model = "Generic Console";
    RAMSizeGb = 2;
    HardDriveSizeGb = 128;
}

public GameConsole(ProcessorType proc, int clock, ManufacturerName man, MediaType med, string model, int
ram, int storage)
{
    Processor = proc;
    CpuClockMHz = clock;
    ManufacturerName = man;
    Media = med;
    Model = model;
    RAMSizeGb = ram;
    HardDriveSizeGb = storage;
}

private GameConsole(GameConsole other) // constructor for cloning
{
    Processor = other.Processor;
    CpuClockMHz = other.CpuClockMHz;
    ManufacturerName = other.ManufacturerName;
    Media = other.Media;

```

```

    Model = other.Model;
    RAMSizeGb = other.RAMSizeGb;
    HardDriveSizeGb = other.HardDriveSizeGb;

    _installedGames = new List<string>(other._installedGames);
    _accounts = new List<string>(other._accounts);
}

private static readonly Random Random = new Random();

public void AddAccount(string name)
{
    _accounts.Add(name);
    _userAdded?.Invoke(this, name);
}

public void ConnectedExternalDrive()
{
    _driveConnected?.Invoke(this, EventArgs.Empty);
}

public void InstallGame(string gameName)
{
    _installedGames.Add(gameName);
    _gameInstalled?.Invoke(this, gameName);
}

public void ReplaceGamepad()
{
    _gamepadReplaced?.Invoke(this, EventArgs.Empty);
}

public bool TrySoftwareHack()
{
    {
        var hackSuccessChances = this.ManufacturerName switch
        {
            ManufacturerName.Microsoft => 30,
            ManufacturerName.Nintendo => 40,
            ManufacturerName.Sega => 70,
            ManufacturerName.Sony => 33,
            ManufacturerName.Valve => 5,
            ManufacturerName.Lenovo => 99,
            _ => 50
        };

        var success = Random.Next(0, 100) < hackSuccessChances;

        Console.WriteLine(success
            ? "Console successfully hacked. Steal everything!"
            : "ERROR: Try to inject virus to console failed. Console called police!");

        return success;
    }
}

```

```

public bool TryHardwareHack()
{
    var hackSuccessChances = this.Media switch
    {
        MediaType.Cartridge => 41,
        MediaType.BluRay    => 30,
        MediaType.CD        => 33,
        MediaType.DVD       => 29,
        MediaType.Flash     => 13,
        _                   => 50
    };

    var success = Random.Next(0, 100) < hackSuccessChances;

    Console.WriteLine(success
        ? "You hacked storage system. Load any game!"
        : "ERROR: Console`s hardware detected your penetration. Leave country as soon as you can!");

    return success;
}

private static readonly string[] AvailableGames =
[
    "GTA 5", "GTA 4", "CS2", "Dota 2", "Everlasting Summer", "League Of Legends", "Minecraft", "Forza Horizon 5",
    "Elden Ring", "Dark Souls 4", "BeamNG.Drive", "The Crew 2", "The Last of Us", "The Legend of Zelda"
];

private const string Alphabet =
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz1234567890_";

public static GameConsole Generate()
{
    var manufacturer = (ManufacturerName)Random.Next(0, 6);

    var model = manufacturer switch
    {
        ManufacturerName.Microsoft => "Xbox " + (Random.Next(0, 2) == 0 ? "Series S" : "Series X"),
        ManufacturerName.Nintendo  => "Nintendo " + (Random.Next(0, 2) == 0 ? "Switch" : "Switch 2"),
        ManufacturerName.Sony      => "Playstation " + Random.Next(1, 5),
        ManufacturerName.Sega      => "Dreamcast",
        ManufacturerName.Valve     => "Steam " + (Random.Next(0, 2) == 0 ? "Deck" : "Machine"),
        ManufacturerName.Lenovo    => "Legion Go",
        _                          => "Unknown"
    };

    var console = new GameConsole(
        (ProcessorType)Random.Next(0, 4),
        Random.Next(800, 4936),
        manufacturer,
        (MediaType)Random.Next(0, 5),
        model,
        Random.Next(1, 16),

```

```

        Random.Next(120, 1024)
    );

    var gamesAmount = Random.Next(0, 4);
    for (var i = 0; i < gamesAmount; i++)
    {
        var game = AvailableGames[Random.Next(0, 8)];

        if (!console.InstalledGames.Contains(game))
            console.InstallGame(game);
    }

    var accountsAmount = Random.Next(1, 3);
    for (var i = 0; i < accountsAmount; i++)
    {
        var accountNameLenght = Random.Next(8, 27);
        var accountName = new StringBuilder();

        for (var j = 0; j < accountNameLenght; j++)
        {
            var randomChar = Alphabet[Random.Next(Alphabet.Length)];
            accountName.Append(randomChar);
        }

        console.AddAccount(accountName.ToString());
    }

    return console;
}

public static List<GameConsole> Generate100()
{
    var consoles = new List<GameConsole>();

    for (var i = 0; i < 100; i++)
    {
        consoles.Add(Generate());
    }

    return consoles;
}

public void ShowAccounts()
{
    for (var i = 0; i < Accounts.Count; i++)
    {
        Console.WriteLine($"{i}. {Accounts[i]}");
    }
}

public void ShowGames()
{
    for (var i = 0; i < InstalledGames.Count; i++)

```

```

    {
        Console.WriteLine($"{i}. {InstalledGames[i]}");
    }
}

public object Clone()
{
    return new GameConsole(this);
}
}

```

StringExtension.cs

```

public static class StringExtension
{
    public static string RemoveLatinLetters(this string input)
    {
        if (string.IsNullOrEmpty(input))
        {
            return input;
        }

        // var sb = new StringBuilder();
        // foreach (var c in input)
        // {
        //     var isLatin = (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z');
        //     if (!isLatin)
        //     {
        //         sb.Append(c);
        //     }
        // }
        var output = new string(input.Where(c => !((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z'))).ToArray());

        return output;
        // return sb.ToString();
    }
}

```

Лабораторная работа №7

Задание:

- 1) Реализовать класс делегат с сигнатурой `delegate double MyDel(int[] a)`.
- 2) Реализовать анонимный метод, вычисляющий разность между наибольшим и наименьшим элементом массива.
- 3) Написать 10 unit тестов для проверки корректности вычислений.
- 4) Для класса Console из прошлого урока опишите реализацию событий:
 1. Добавление нового пользователя;
 2. Подключение внешнего накопителя;
 3. Установка новой игры;
 4. Замена геймпада;
 5. Для каждого события реализовать `add`, `remove`.
- 5) Запустите Unit тесты из задания 1 и 2 используя утилиту `nunit-console: dotnet test Tests.dll --test-adapter-path NUnit3.TestAdapter.dll`

Код:

```
MyDel.cs

public delegate double MyDel(int[] a);

ArrayDifference.cs

public static class ArrayDifference
{
    public static MyDel Create()
    {
        return delegate(int[] a)
        {
            if (a.Length == 0)
            {
                throw new ArgumentException();
            }

            var min = a[0];
            var max = a[0];

            foreach (var item in a)
            {
                if (item < min)
```

```

        min = item;

        if (item > max)
            max = item;
    }

    return max - min;
};
}
}

```

DifferenceTests.cs

```

public class DifferenceTests
{
    private MyDel _difference;

    [SetUp]
    public void Setup()
    {
        _difference = ArrayDifference.Create();
    }

    [TestCase(new int[] {1, 2, 3, 4, 5, 6, 7, 8}, 7)]
    [TestCase(new int[] {6, 23, 3, 4, 5, 6, 7, 8}, 20)]
    [TestCase(new int[] {6, 0, 3, 4, 10, 6, 7, 1}, 10)]
    [TestCase(new int[] {5}, 0)]
    [TestCase(new int[] {5, 5, 5, 5, 5}, 0)]
    [TestCase(new int[] {-10, -15, -20, -25, -30}, 20)]
    [TestCase(new int[] {-10, 0, -20, 30, 40}, 60)]
    [TestCase(new int[] {0, 0, 0, 0}, 0)]
    [TestCase(new int[] {-1, 0, 0, 0}, 1)]
    public void Test_Normal(int[] array, double result)
    {
        Assert.That(_difference.Invoke(array), Is.EqualTo(result));
    }

    [Test]
    public void Test_Empty()
    {
        int[] array = [];
        Assert.Throws<ArgumentException>(() => _difference.Invoke(array));
    }
}

```

Лабораторная работа №8

Задание:

- 1) Реализовать класс, имеющий:
 1. Public, Protected, Private поля;
 2. Public, Protected, Private Свойства;
 3. Public, Protected, Private Методы;
 4. Базовый конструктор;
 5. Конструктор с полями;
 6. Поместить его в Dll.
- 2) С помощью рефлексии вывести информация о:
 1. Всех полях;
 2. Всех методах;
 3. Всех свойствах.
- 3) С помощью рефлексии:
 1. Прочитать значения полей и свойств;
 2. Задать значения полей и свойств;
 3. Вызвать методы;
 4. Вызвать конструкторы;
 5. Загрузить библиотеку и получить информацию о классе.
- 4) Прочитать про паттерн Singleton (<https://github.com/StepanovPetr/object-oriented-programming/tree/master/Patterns/Creational%20Patterns>) (<https://www.dofactory.com/net/singleton-design-pattern>).
- 5) Смоделировать факультеты ОмГТУ (Продумать что должно быть в базовом классе, интерфейсы).
- 6) Реализовать для каждого факультета (Института) отдельный класс, сделать его Singleton.
- 7) Прочитать про паттерн Prototype (<https://github.com/StepanovPetr/object-oriented-programming/tree/master/Patterns/Creational%20Patterns/Prototype>) (<https://www.dofactory.com/net/prototype-design-pattern>) (<https://docs.microsoft.com/en-us/dotnet/api/system.icloneable.clone?view=netframework-4.8>)
- 8) Для класса Console из урока 6, реализовать интерфейс ICloneable.

Код:

```
IFaculty.cs
public interface IFaculty
{
    string Name { get; }
    string Dean { get; }
```

```

int Departments { get; }
int StudentCount { get; } // Добавим количество студентов
void PrintInfo();
double CalculateYearlyBudget(); // Метод для расчета бюджета
}

```

FacultyBase.cs

```

public abstract class FacultyBase : IFaculty
{
    public string Name { get; protected set; }
    public string Dean { get; protected set; }
    public int Departments { get; protected set; }
    public int StudentCount { get; protected set; }

    protected FacultyBase(string name, string dean, int departments, int students)
    {
        Name = name;
        Dean = dean;
        Departments = departments;
        StudentCount = students;
    }

    public virtual void PrintInfo()
    {
        Console.WriteLine($"--- Faculty: {Name} ---");
        Console.WriteLine($"Dean: {Dean} | Depts: {Departments} | Students: {StudentCount}");
    }

    public abstract double CalculateYearlyBudget();

    public virtual void HoldConference()
    {
        Console.WriteLine($"{Name} is holding a general academic conference.");
    }
}

```

FIFaculty.cs

```

public sealed class FIFaculty : FacultyBase
{
    private static readonly Lazy<FIFaculty> _instance = new Lazy<FIFaculty>(() => new FIFaculty());
    public static FIFaculty Instance => _instance.Value;

    public int ComputerLabs { get; private set; }

    private FIFaculty() : base(
        name: "ФИТиКС (Факультет информационных технологий и компьютерных систем)",
        dean: "Грицай А. С.",
        departments: 6,
        students: 1200
    )
    {
        ComputerLabs = 15;
    }

    public override double CalculateYearlyBudget()
    {
        double baseRate = 50000;
        double labMaintenance = 150000;
        return (StudentCount * baseRate) + (ComputerLabs * labMaintenance);
    }
}

```

```

public override void PrintInfo()
{
    base.PrintInfo();
    Console.WriteLine($"Tech Infrastructure: {ComputerLabs} specialized computer labs.");
}

public void OrganizeHackathon()
{
    Console.WriteLine($"[EVENT] {Name} is starting a 48-hour hackathon for {StudentCount} students!");
}
}

FTNGFaculty.cs

public sealed class FTNGFaculty : FacultyBase
{
    private static readonly Lazy<FTNGFaculty> _instance = new Lazy<FTNGFaculty>(() => new FTNGFaculty());
    public static FTNGFaculty Instance => _instance.Value;

    public int DrillingSimulators { get; private set; }
    public int PartnerCompanies { get; private set; }

    private FTNGFaculty() : base(
        name: "ФТНГ (Факультет транспорта нефти и газа)",
        dean: "Квасов И. Н.",
        departments: 6,
        students: 850
    )
    {
        DrillingSimulators = 4;
        PartnerCompanies = 12;
    }

    public override double CalculateYearlyBudget()
    {
        double studentSubsidy = 60000;
        double simulatorMaintenance = 500000;
        double partnerGrants = PartnerCompanies * 200000;

        return (StudentCount * studentSubsidy) + (DrillingSimulators * simulatorMaintenance) + partnerGrants;
    }

    public override void PrintInfo()
    {
        base.PrintInfo();
        Console.WriteLine($"Industrial Stats: {DrillingSimulators} heavy simulators | {PartnerCompanies} partner companies.");
    }

    public void StartFieldPractice()
    {
        Console.WriteLine($"[FIELD] Students of {Name} are heading to the oil fields for practical training.");
    }
}

RTFFaculty.cs

public sealed class RTFFaculty : FacultyBase
{
    private static readonly Lazy<RTFFaculty> _instance = new Lazy<RTFFaculty>(() => new RTFFaculty());
    public static RTFFaculty Instance => _instance.Value;

    public int LabStations { get; private set; }

```

```

private RTFFaculty() : base(
    name: "РТФ (Радиотехнический факультет)",
    dean: "Кропачев Д. Ю.",
    departments: 5,
    students: 700
)
{
    LabStations = 24;
}

public override double CalculateYearlyBudget()
{
    double studentBase = 45000;
    double equipmentCalibration = LabStations * 25000;

    return (StudentCount * studentBase) + equipmentCalibration;
}

public override void PrintInfo()
{
    base.PrintInfo();
    Console.WriteLine($"Hardware Stats: {LabStations} measurement stations");
}

public void CalibrateRadioEquipment()
{
    Console.WriteLine($"[MAINTENANCE] {Name} is calibrating oscilloscopes and signal generators...");
}
}

```

Building.cs

```

public class Building
{
    public string address;
    protected int securityCode;
    private double _vaultBalance;

    public string Name { get; set; }
    protected double Height { get; set; }
    private int Residents { get; set; }

    private static readonly Random Random = new Random();

    public Building()
    {
        Name = "Anonymous building";
        Height = 0.0;
        Residents = 0;
        address = "Unknown";
        securityCode = 1234;
        _vaultBalance = 0.0;
    }

    public Building(string name, double height, int residents, string addr, double vaultBalance)
    {
        Name = name;
        Height = height;
        Residents = residents;
        address = addr;
        securityCode = Random.Next(1000, 9999);
    }
}

```

```

    _vaultBalance = vaultBalance;
}

protected void CheckSecurity()
{
    Console.WriteLine($"We are checking. Current security code {securityCode}.");
}

private void TriggerAlarm()
{
    Console.WriteLine($"Woop, woop, that's the sound of the police.");
}

public bool Demolish()
{
    if (Height > 0.0 && Residents > 0) {
        var chance = Random.NextDouble() * 100;
        var strength = Height * 0.5;

        if (chance > strength)
        {
            Console.WriteLine($"Building {Name} was demolished.");

            Height = 0;
            Residents = 0;

            return true;
        }
        else
        {
            var diedResidents = Random.Next(0, Residents);

            Console.WriteLine($"Building {Name} still alive. Died {diedResidents} residents.");

            Residents -= diedResidents;

            return false;
        }
    }
    else
    {
        Console.WriteLine($"Building {Name} already not here.");

        return false;
    }
}

public bool Rob()
{
    Console.WriteLine($"Trying to rob building {Name} at {address}.");

    TriggerAlarm();

    if (_vaultBalance > 0)
    {
        var chance = Random.NextDouble();

        if (chance >= 0.5)
        {
            Console.WriteLine($"Building {Name} was robbed.");
        }
    }
}

```

```

        _vaultBalance = 0;

        return true;
    }
    else
    {
        Console.WriteLine($"Building {Name} wasn't robbed.");

        return false;
    }
}
else
{
    Console.WriteLine($"Building {Name} not contains any balance.");

    return false;
}
}
}

```

Program.cs

```

using lab8_lib;

public class Program
{
    static void Main()
    {
        var building = new Building();

        Console.WriteLine($"Building: {building.Name} at {building.address}.");

        var type = typeof(Building);

        Console.WriteLine($"\\n{type.FullName}"); // инфо о классе

        // поля
        Console.WriteLine("\\nFields before:");
        foreach (var field in type.GetFields())
        {
            Console.WriteLine($"{{field.FieldType.Name}} {{field.Name}} {{field.GetValue(building)}}");
        }

        var fieldAddress = type.GetField("address");

        fieldAddress?.SetValue(building, "Not here");

        Console.WriteLine($"\\nField after:\\n{{fieldAddress?.FieldType.Name}} {{fieldAddress?.Name}}
{{fieldAddress?.GetValue(building)}}");

        // свойства
        Console.WriteLine("\\nProperties before:");
        foreach (var property in type.GetProperties())
        {
            Console.WriteLine($"{{property.PropertyType.Name}} {{property.Name}} {{property.GetValue(building)}}");
        }

        var propertyName = type.GetProperty("Name");

        propertyName?.SetValue(building, "Fufelschmerz State Building");

        Console.WriteLine($"\\nProperty after:\\n{{propertyName?.PropertyType.Name}} {{propertyName?.Name}} {{prop-

```

```

ertyName?.GetValue(building)}");

    // методы
    Console.WriteLine("\nMethods:");
    foreach (var method in type.GetMethods())
    {
        Console.WriteLine($"{method.ReturnType.Name} {method.Name}");
    }

    var obj = Activator.CreateInstance(type); // использование конструктора
    Console.WriteLine($"{type.GetMethod("Demolish")?.Invoke(obj, null)}"); // вызов метода (просто пункты
    // рядом в одно решение уместились)
}
}

```

Лабораторная работа №9

Задание:

- 1) Переписать задание из урока 5 так, чтобы:
- 2) Поиск в каждом диске происходил в отдельном потоке.
- 3) Поиск в каждой папке корневой директории происходил в отдельном потоке.
- 4) Прочитать про паттерн Singleton (<https://github.com/StepanovPetr/object-oriented-programming/tree/master/Patterns/Creational%20Patterns>) (<https://www.dofactory.com/net/singleton-design-pattern>).
- 5) Смоделировать факультеты ОмГТУ (Продумать что должно быть в базовом классе, интерфейсы).
- 6) Реализовать для каждого факультета (Института) отдельный класс, сделать его Singleton. (потокобезопасный).

Код:

Task3.cs

```
public class Task3
{
    public void Execute()
    {
        Console.WriteLine("Введите путь к исходной папке: ");
        string sourcePath = Console.ReadLine();

        Console.WriteLine("Введите путь к папке бэкапов: ");
        string backupRoot = Console.ReadLine();

        DirectoryInfo source = new DirectoryInfo(sourcePath);
        DirectoryInfo backup = new DirectoryInfo(backupRoot);

        if (!source.Exists)
        {
            Console.WriteLine("Исходная папка не найдена");
            return;
        }

        if (!backup.Exists)
            backup.Create();

        int latestVersion = GetLatestVersion(backup);

        if (latestVersion == 0)
        {
            CreateBackup(source, backup, 1);
            Console.WriteLine("Создана версия бэкапа 1");
            return;
        }

        DirectoryInfo latestBackupFolder = new DirectoryInfo(Path.Combine(backup.FullName, "v" + latestVersion));

        bool changed = IsChanged(source, latestBackupFolder);
    }
}
```

```

    if (changed)
    {
        int newVersion = latestVersion + 1;
        CreateBackup(source, backup, newVersion);
        Console.WriteLine("Создана версия бэкапа " + newVersion);
    }
    else
    {
        Console.WriteLine("Изменения не обнаружены");
    }
}

private int GetLatestVersion(DirectoryInfo backup)
{
    int max = 0;
    foreach (DirectoryInfo dir in backup.GetDirectories())
    {
        if (dir.Name.StartsWith("v"))
        {
            if (int.TryParse(dir.Name.Substring(1), out int v))
            {
                if (v > max)
                    max = v;
            }
        }
    }

    return max;
}

private bool IsChanged(DirectoryInfo source, DirectoryInfo backupFolder)
{
    FileInfo[] srcFiles = source.GetFiles(".*", SearchOption.AllDirectories);
    FileInfo[] bakFiles = backupFolder.GetFiles(".*", SearchOption.AllDirectories);

    if (srcFiles.Length != bakFiles.Length)
        return true;

    foreach (FileInfo src in srcFiles)
    {
        string relative = Path.GetRelativePath(source.FullName, src.FullName);
        string bakPath = Path.Combine(backupFolder.FullName, relative);
        FileInfo bak = new FileInfo(bakPath);

        if (!bak.Exists)
            return true;

        string md5Src = GetMD5(src.FullName);
        string md5Bak = GetMD5(bak.FullName);

        if (!md5Src.Equals(md5Bak, StringComparison.OrdinalIgnoreCase))
            return true;
    }

    return false;
}

private void CreateBackup(DirectoryInfo source, DirectoryInfo backup, int version)
{
    string versionFolder = Path.Combine(backup.FullName, "v" + version);
    DirectoryInfo target = new DirectoryInfo(versionFolder);
    target.Create();
}

```

```

foreach (FileInfo file in source.GetFiles("*", SearchOption.AllDirectories))
{
    string relative = Path.GetRelativePath(source.FullName, file.FullName);
    string destPath = Path.Combine(target.FullName, relative);
    Directory.CreateDirectory(Path.GetDirectoryName(destPath));
    file.CopyTo(destPath, true);
}
}

private string GetMD5(string path)
{
    using (var md5 = MD5.Create())
    using (var stream = File.OpenRead(path))
    {
        byte[] hash = md5.ComputeHash(stream);
        StringBuilder sb = new StringBuilder();
        foreach (byte b in hash)
            sb.Append(b.ToString("x2"));
        return sb.ToString();
    }
}
}

```

Лабораторная работа №10

Задание:

- 1) Разработайте консольное приложение, которое имитирует обработку большого набора аудио.
- 2) Разделите набор аудио на несколько частей (на количество процессоров физических) и обработайте каждую часть в отдельной задаче. Используйте Task.Run для запуска задач.
- 3) В каждой задаче выполните какую-либо обработку аудио (имитировать что-то из: удаление шума, ускорение воспроизведение, применение фильтров, конвертация формата).
- 4) Используйте Task.WaitAll для ожидания завершения всех задач.
- 5) После завершения всех задач сохраните обработанные аудио в отдельную папку.
- 6) Выведите сообщения в консоль о завершении обработки каждого аудио и общем результате.
- 7) Реализуйте механизм отмены обработки (например, с помощью CancellationTokenSource). Чтобы можно было отменить во время выполнения таска по завершении аудио, которое в момент отмены обрабатывалось.
- 8) Добавьте обработку исключений для каждого аудио отдельно.

Код:

```
Audio.cs
public class Audio
{
    public string SourcePath { get; }
    public string FileName { get; }
    public byte[] Data { get; private set; }

    public Audio(string sourcePath)
    {
        SourcePath = sourcePath ?? throw new ArgumentNullException(nameof(sourcePath));
        FileName = Path.GetFileName(sourcePath);
        Data = File.ReadAllBytes(sourcePath);
    }

    public void SlowDownBy2X(CancellationToken token)
    {
        token.ThrowIfCancellationRequested();

        Thread.Sleep(1000);

        var source = Data;
        var slowed = new byte[source.Length * 2];

        Buffer.BlockCopy(source, 0, slowed, 0, source.Length);
    }
}
```

```

        Buffer.BlockCopy(source, 0, slowed, source.Length, source.Length);

        Data = slowed;
    }

    public void Save(string outputDirectory)
    {
        // Directory.CreateDirectory(outputDirectory);
        var targetPath = Path.Combine(outputDirectory, FileName);
        File.WriteAllBytes(targetPath, Data);
    }
}

Program.cs

internal static class Program
{
    private static void Main()
    {
        Console.WriteLine("Enter the path to the folder with audio files:");
        var inputFolder = Console.ReadLine();

        if (string.IsNullOrEmpty(inputFolder) || !Directory.Exists(inputFolder))
        {
            Console.WriteLine("The folder does not exist.");
            return;
        }

        var files = Directory.EnumerateFiles(inputFolder)
            .Where(IsAudioFile)
            .ToList();

        if (files.Count == 0)
        {
            Console.WriteLine("No audio files were found.");
            return;
        }

        var outputFolder = Path.Combine(inputFolder, "ProcessedAudio");
        var coreCount = Math.Max(1, GetPhysicalCoreCount());
        var workerCount = Math.Min(coreCount, files.Count);
        // var chunks = SplitIntoChunks(files, workerCount).ToList();
        var chunks = SplitIntoChunks(files, 2).ToList();

        Console.WriteLine($"Found {files.Count} audio files.");
        Console.WriteLine($"Using {workerCount} tasks.");
        Console.WriteLine("Press C to cancel after the current audio finishes.");

        using var cts = new CancellationTokenSource();
        var processedAudios = new ConcurrentBag<Audio>();

        var tasks = chunks.Select((chunk, index) =>
            Task.Run(() => ProcessChunk(chunk, index + 1, processedAudios, cts.Token), cts.Token))
            .ToArray();

        var cancellationMonitor = Task.Run(() => MonitorCancellation(cts), cts.Token);

        try
        {
            Task.WaitAll(tasks, cts.Token);
        }
        catch (OperationCanceledException)
    }
}

```

```

    {
        Console.WriteLine("Processing was cancelled.");
    }
    catch (AggregateException ex)
    {
        foreach (var inner in ex.Flatten().InnerExceptions)
        {
            if (inner is not OperationCanceledException)
            {
                Console.WriteLine($"Task error: {inner.Message}");
            }
        }
    }
}

cts.Cancel();
try
{
    cancellationMonitor.Wait(cts.Token);
}
catch (OperationCanceledException)
{
    Console.WriteLine("Processing was cancelled.");
}

Directory.CreateDirectory(outputFolder);

foreach (var audio in processedAudios.OrderBy(a => a.FileName, StringComparer.OrdinalIgnoreCase))
{
    try
    {
        audio.Save(outputFolder);
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Failed to save {audio.FileName}: {ex.Message}");
    }
}

Console.WriteLine($"Processing completed. Saved {processedAudios.Count} files to: {outputFolder}");
}

private static void ProcessChunk(
    IEnumerable<string> chunk,
    int taskNumber,
    ConcurrentBag<Audio> processedAudios,
    CancellationToken token)
{
    foreach (var path in chunk)
    {
        if (token.IsCancellationRequested)
        {
            Console.WriteLine($"Task {taskNumber}: cancellation requested, stopping after the current audio.");
            break;
        }

        try
        {
            var audio = new Audio(path);
            audio.SlowDownBy2X(token);
            processedAudios.Add(audio);
            Console.WriteLine($"Task {taskNumber}: completed {audio.FileName}");
        }
    }
}

```

```

    }
    catch (OperationCanceledException)
    {
        Console.WriteLine($"Task {taskNumber}: cancelled {Path.GetFileName(path)}");
        break;
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Task {taskNumber}: failed to process {Path.GetFileName(path)} - {ex.Message}");
    }
}

private static void MonitorCancellation(CancellationTokenSource cts)
{
    try
    {
        while (!cts.IsCancellationRequested)
        {
            if (!Console.KeyAvailable)
            {
                Thread.Sleep(100);
                continue;
            }

            var key = Console.ReadKey(true).Key;
            if (key == ConsoleKey.C)
            {
                Console.WriteLine("Cancellation requested.");
                cts.Cancel();
                break;
            }
        }
    }
    catch
    {
        // ignored
    }
}

private static List<List<string>>> SplitIntoChunks(List<string> items, int workers)
{
    var result = new List<List<string>>>();

    for (var i = 0; i < workers; i++)
    {
        result.Add([]);
    }

    for (var i = 0; i < items.Count; i++)
    {
        result[i % workers].Add(items[i]);
    }

    return result;
}

private static bool IsAudioFile(string path)
{
    var ext = Path.GetExtension(path);

```

```
return ext.Equals(".wav", StringComparison.OrdinalIgnoreCase)
    || ext.Equals(".mp3", StringComparison.OrdinalIgnoreCase)
    || ext.Equals(".flac", StringComparison.OrdinalIgnoreCase)
    || ext.Equals(".aac", StringComparison.OrdinalIgnoreCase)
    || ext.Equals(".ogg", StringComparison.OrdinalIgnoreCase)
    || ext.Equals(".m4a", StringComparison.OrdinalIgnoreCase);
}

private static int GetPhysicalCoreCount()
{
    return Environment.ProcessorCount;
}
}
```

Заключение

В ходе выполнения цикла лабораторных работ были систематизированы и закреплены практические навыки объектно-ориентированного программирования на языке C#. В процессе работы освоено проектирование и реализация классов с использованием инкапсуляции, наследования и полиморфизма, а также разработка интерфейсов и их реализация в пользовательских классах. Закреплены навыки работы с коллекциями, включая применение LINQ-запросов для фильтрации, сортировки и проекции данных. Изучены механизмы создания и использования делегатов, анонимных методов и событий с управлением подпиской, а также написания unit-тестов для проверки корректности логики. Освоено применение рефлексии для анализа метаданных типов и доступа к полям, свойствам, методам и конструкторам во время выполнения. Практически реализованы порождающие паттерны проектирования Singleton и Prototype. Отдельное внимание уделено многопоточному программированию: созданию и синхронизации задач, разделению нагрузки между ядрами процессора, ожиданию завершения группы задач, управлению асинхронными операциями с использованием механизма отмены, а также обработке исключений в многопоточной среде. Дополнительно освоено создание методов расширения для встроенных типов. Все разработанные программные компоненты протестированы и демонстрируют корректную работу, а полученные компетенции являются надёжной основой для дальнейшего изучения платформы .NET и разработки сложных прикладных систем.